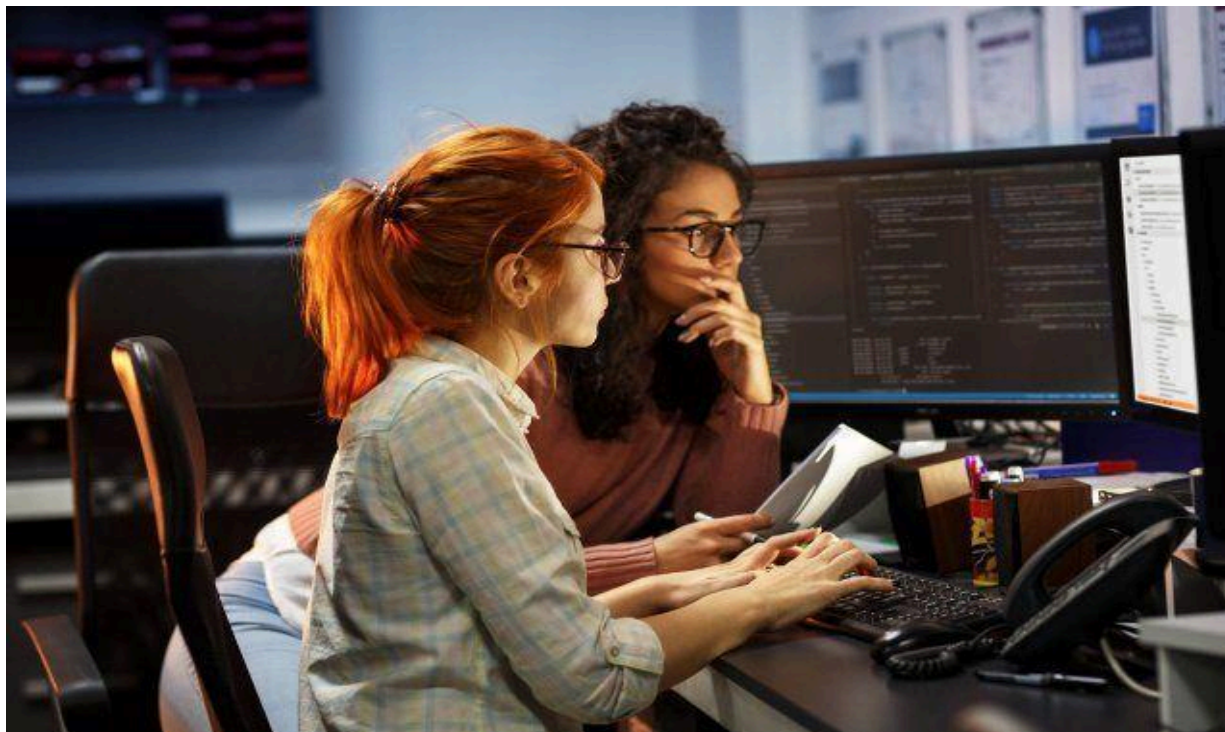




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 1

Tema: Programación y Principios de Diseño Orientado a Objetos.

SEMANA 5

Autor:

Introducción:

Bienvenidos al material de lectura #5 de la Unidad #1 de la materia Programación Orientada a Objetos. En este documento exploramos sobre modelado y Diseño de Software Orientado a Objetos.

Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en las actividades propuestas, ya que contribuirán significativamente a tu proceso de aprendizaje.

1.9. Modelado y Diseño de Software Orientado a Objetos

El **Modelado y Diseño Orientado a Objetos** es la fase creativa y estratégica en la construcción de software. Antes de escribir una sola línea de código, nos detenemos a pensar y a planificar. El objetivo es crear un "plano" o "mapa" de nuestro futuro programa.

Analogía: Imagina que quieres construir un coche de LEGOs. No empiezas a juntar piezas al azar. Primero, miras el manual: ves qué piezas necesitas, cómo se conectan y cuál será el resultado final. En software, ese manual es nuestro diseño.

En la Programación Orientada a Objetos (POO), este proceso se centra en identificar los **objetos** clave del problema. Un objeto es una entidad que tiene **estado** (datos o atributos) y **comportamiento** (acciones o métodos).

- **Estado:** ¿Qué *sabe* o *tiene* el objeto? (Ej: Un objeto **Coche** tiene un **color**, una **marca** y una **velocidadActual**).
- **Comportamiento:** ¿Qué *hace* el objeto? (Ej: Un **Coche** puede **acelerar()**, **frenar()** y **encender()**).

El diseño nos ayuda a definir las "plantillas" para estos objetos, que en Java llamamos **clases**.

1.9.1. Introducción a UML (Lenguaje Unificado de Modelado)

UML es el lenguaje estándar para dibujar esos "planos" de software. Nos proporciona un conjunto de símbolos y diagramas para que todos los desarrolladores puedan

¿Te ha gustado lo que has leído hasta ahora?

Anota a continuación los aspectos que consideras más importantes de esta sección:

[illegible]

entender el diseño de un sistema, sin importar el lenguaje de programación que usen.

3.1.1. Diagramas de Clases y Casos de Uso

Diagrama de Clases: La Estructura Interna

Este es el diagrama más importante en POO. Muestra la estructura estática del sistema, es decir, las clases y cómo se relacionan entre sí. Es el esqueleto de tu aplicación.

Ejemplo Básico:

Imaginemos una clase **Usuario** en Java.

```
public class Usuario {  
    private String nombreUsuario;  
    private String contraseña;  
    private int edad;  
  
    public boolean iniciarSesion(String pass) {  
        return this.contrasena.equals(pass);  
    }  
  
    public void cambiarContraseña(String nuevaContraseña) {  
        this.contrasena = nuevaContraseña;  
    }  
}
```

Representación en Diagrama de Clases UML:

El diagrama sería un rectángulo con tres partes:

Usuario



- El signo - significa **private** (privado).
- El signo + significa **public** (público).

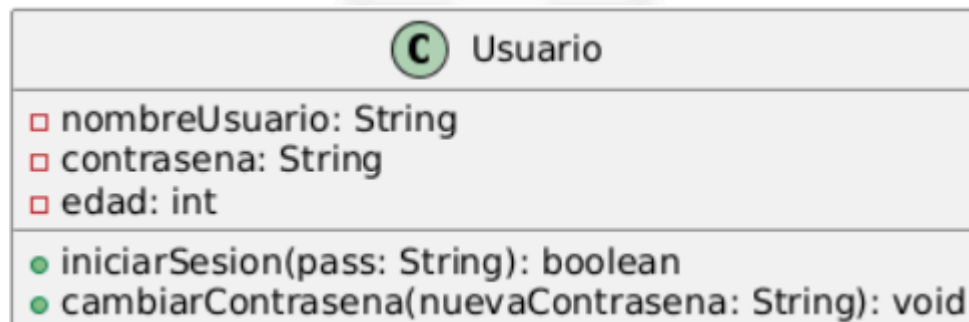


Diagrama de Casos de Uso: La Funcionalidad Externa

En términos más sencillos, un caso de uso responde a la pregunta: "**¿Qué puede hacer un usuario con el sistema?**". No describe *cómo* el sistema lo hace por dentro (la lógica de programación), sino *qué* funcionalidad ofrece desde la perspectiva del usuario.

Analogía simple: Piensa en un cajero automático (ATM). 

- **Caso de uso:** "Retirar dinero".
- **Actor:** El "Cliente del banco".
- **Objetivo:** El cliente quiere obtener efectivo de su cuenta.
- **Interacción:** El cliente inserta la tarjeta, introduce el PIN, selecciona el monto, y el sistema dispensa el dinero y devuelve la tarjeta.

El caso de uso "Retirar dinero" captura toda esa secuencia de pasos para lograr el objetivo del cliente.

Componentes Clave

1. **Actor:** Es quien o qué inicia el caso de uso. No tiene por qué ser una persona; puede ser otro sistema de software o incluso un evento de tiempo. Se representa con una figura de palo. 🧑
 2. **Caso de Uso:** Es la funcionalidad o meta que el actor quiere lograr. Se representa con un óvalo.
 3. **Sistema:** Es el software que estás diseñando. Se representa con una caja que contiene los casos de uso.
-

¿Para qué sirve?

- **Definir el alcance:** Ayuda a establecer qué hará (y qué no hará) el sistema.
- **Comunicación:** Sirve como un lenguaje común entre los clientes, los desarrolladores y los analistas para entender los requisitos del software.
- **Guía para el desarrollo:** Cada caso de uso se puede convertir en una guía para diseñar, programar y probar una parte del sistema.

Este diagrama describe lo que el sistema *hace* desde la perspectiva del usuario (el "actor"). Es perfecto para entender los requisitos de un proyecto. Responde a la pregunta: ¿Para qué sirve este software?

Ejemplo Básico:

Pensemos en un sistema de blog simple.

- **Actores:** "Escritor", "Lector".
- **Casos de Uso:** "Publicar Artículo", "Comentar Artículo", "Leer Artículo".

El diagrama mostraría al "Escritor" conectado con "Publicar Artículo". Tanto el "Escritor" como el "Lector" estarían conectados con "Leer Artículo" y "Comentar Artículo".

Este diagrama no muestra *cómo* se programa, sino *qué* se debe poder hacer.

3.1.2. Relación entre Clases: Asociación, Herencia, Composición

Estas relaciones son la clave para construir sistemas complejos a partir de clases simples.

Herencia (Relación "es un")

La herencia permite crear una nueva clase (subclase) que reutiliza, extiende o modifica el comportamiento de una clase existente (superclase). Es la implementación directa del principio de reutilización.

- Se representa con una flecha de punta hueca.

Ejemplo: Un **Estudiante** y un **Profesor** "son un" tipo de **Persona**. Ambos comparten atributos como **nombre** y **apellido**.

// Superclase o Clase Padre

```
public class Persona {  
    protected String nombre;  
    protected String apellido;  
  
    public void mostrarNombreCompleto() {  
        System.out.println(nombre + " " + apellido);  
    }  
}
```

// Subclase o Clase Hija

```
public class Estudiante extends Persona {
```

```
private String idEstudiante;

public void estudiar() {
    System.out.println(nombre + " está estudiando.");
}
}
```

En UML, una flecha iría desde **Estudiante** hasta **Persona**.

Asociación (Relación "usa un")

Indica que dos clases están conectadas y pueden interactuar, pero sus ciclos de vida no están ligados. Una puede existir sin la otra.

- Se representa con una línea sólida.

Ejemplo: Un **Conductor** "conduce un" **Vehiculo**. El **Conductor** puede existir sin el **Vehiculo** y viceversa.

```
public class Vehiculo {
    private String matricula;
    // ... otros atributos
}

public class Conductor {
    private String nombre;
    private Vehiculo vehiculoAsignado; // El conductor tiene
    una referencia a un vehículo

    public void asignarVehiculo(Vehiculo v) {
        this.vehiculoAsignado = v;
    }
}
```

```
}  
  
}
```

Composición (Relación "es parte de")

Es una forma *fuerte* de asociación donde un objeto (la parte) no puede existir sin el otro (el todo). Si el "todo" se elimina, la "parte" también.

- Se representa con una línea con un rombo relleno del lado del "todo".

Ejemplo: Un **Motor** es "parte de" un **Coche**. El motor no tiene sentido por sí solo, depende del coche para existir.

```
public class Motor {  
    private int caballosDeFuerza;  
  
    public Motor(int hp) {  
        this.caballosDeFuerza = hp;  
    }  
}
```

```
public class Coche {  
    private String marca;  
  
    // El objeto Motor es creado y gestionado DENTRO de la  
    // clase Coche.  
  
    private final Motor motor;  
  
    public Coche(String marca, int hp) {  
        this.marca = marca;  
  
        // El motor se crea cuando se crea el coche. No puede  
        // existir fuera de él.
```



```
        this.motor = new Motor(hp);
    }
}
```

Si se destruye una instancia de **Coche**, su instancia de **Motor** también se pierde.

1.9.2. Uso de Herramientas Modernas

PlantUML: Código para tus Diagramas

PlantUML es genial porque tratas tus diagramas como si fueran código. Puedes versionarlos con Git, revisarlos y colaborar fácilmente.

Ejemplo Básico de PlantUML (para el diagrama **Persona/Estudiante**):

```
@startuml
skinparam classAttributeIconSize 0

class Persona {
    # nombre: String
    # apellido: String
    + mostrarNombreCompleto(): void
}

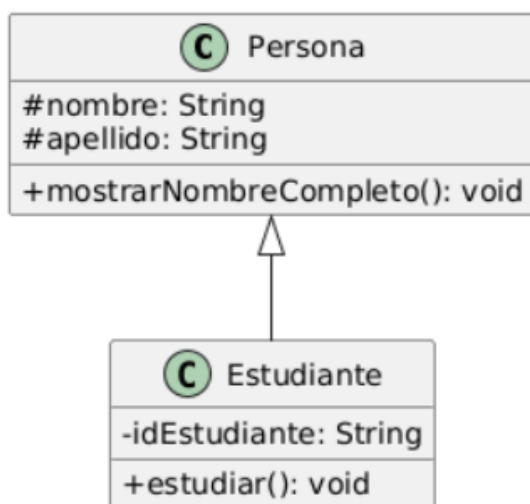
class Estudiante {
    - idEstudiante: String
    + estudiar(): void
}

' Esto define la relación de herencia
```

```
Persona <|-- Estudiante
```

```
@endum1
```

Este simple texto genera una imagen profesional del diagrama de herencia. Herramientas como el plugin de VS Code te permiten visualizarlo al instante.



Lucidchart: Dibuja y Arrastra

Lucidchart es una plataforma web visual. Es como tener una pizarra digital.

Cómo empezar:

1. Crea una cuenta gratuita en Lucidchart.
2. Crea un nuevo documento y selecciona la biblioteca de formas de UML.
3. Arrastra las formas (clases, actores) al lienzo y conéctalas con las líneas de relación correspondientes.
4. Puedes exportar tu diagrama como una imagen (PNG, JPG) para añadirlo a tu documentación.

Es la herramienta perfecta para sesiones de brainstorming y para quienes prefieren un enfoque más gráfico.

GitHub README: Documenta tu Proyecto

El archivo `README.md` es la portada de tu proyecto en GitHub. Un buen `README` explica qué hace tu proyecto y cómo está diseñado.

Cómo incluir tus diagramas:

1. **Si usas Lucidchart:** Exporta tu diagrama como una imagen (ej. `diagrama-clases.png`). Sube esa imagen a tu repositorio de GitHub.
2. **Si usas PlantUML:** Puedes generar la imagen y subirla, o usar servicios que renderizan código PlantUML directamente.

Luego, en tu archivo `README.md`, puedes insertar la imagen usando la sintaxis de Markdown:

Diagrama de Clases

A continuación se muestra la estructura principal de clases de la aplicación:

`![Diagrama de Clases del Proyecto](diagrama-clases.png)`

Diagrama de Clases

A continuación se muestra la estructura principal de clases de la aplicación:

 Diagrama de Clases del Proyecto

Esto hará que la imagen de tu diagrama aparezca directamente en la página principal de tu repositorio, haciendo que tu proyecto se vea mucho más profesional y sea más fácil de entender.